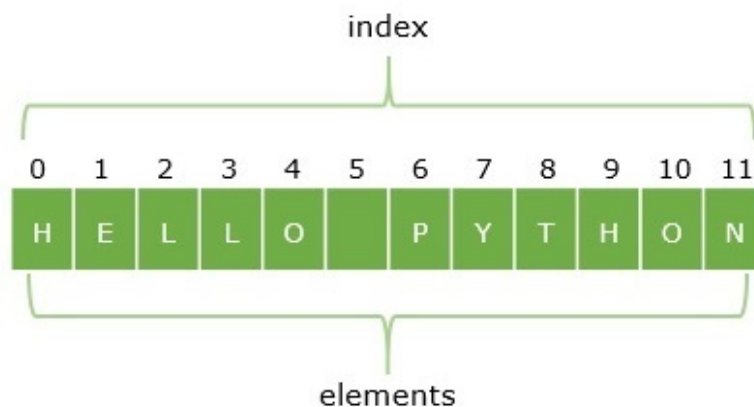


Python Slicing Strings

Python **String slicing** is a way of creating a sub-string from a given string. In this process, we extract a portion or piece of a string. Usually, we use the slice operator "[:]" to perform slicing on a Python String. Before proceeding with string slicing let's understand string indexing.

In Python, a **string** is an ordered sequence of **Unicode characters**. Each character in the string has a unique index in the sequence. The index starts with 0. First character in the string has its positional index 0. The index keeps incrementing towards the end of string.

If a string variable is declared as `var="HELLO PYTHON"`, index of each character in the string is as follows –



Python String Indexing

Python allows you to access any individual character from the string by its index. In this case, 0 is the lower bound and 11 is the upper bound of the string. So, `var[0]` returns H, `var[6]` returns P. If the index in square brackets exceeds the upper bound, Python raises `IndexError`.

Example

In the below example, we accessing the characters of a string through index.

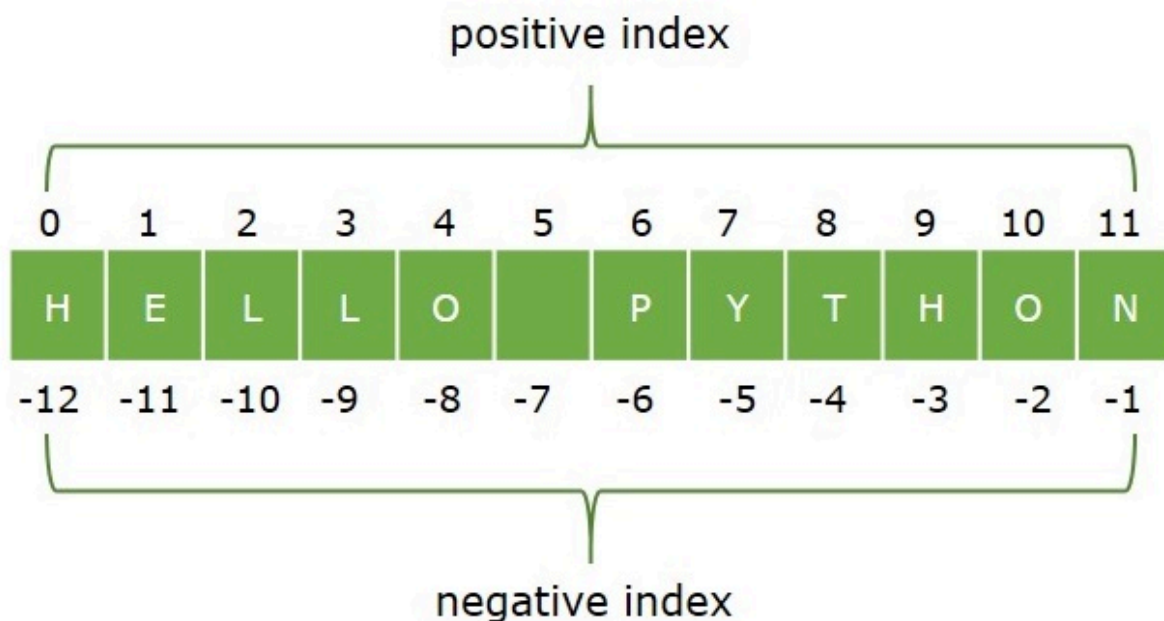
```
var = "HELLO PYTHON"  
print(var[0])  
print(var[7])  
print(var[11])  
print(var[12])
```

On running the code, it will produce the following output –

```
H  
Y  
N  
ERROR!  
Traceback (most recent call last):  
  File "<main.py>", line 5, in <module>  
IndexError: string index out of range
```

Python String Negative & Positive Indexing

One of the unique features of Python sequence types (and therefore a string object) is that it has a negative indexing scheme also. In the example above, a positive indexing scheme is used where the index increments from left to right. In case of negative indexing, the character at the end has -1 index and the index decrements from right to left, as a result the first character H has -12 index.



Example

Let us use negative indexing to fetch N, Y, and H characters.

```
var = "HELLO PYTHON"
print(var[-1])
print(var[-5])
print(var[-12])
```

On executing the above code, it will give the following result –

```
N
Y
H
```

We can therefore use positive or negative index to retrieve a character from the string.

In Python, string is an immutable object. The object is immutable if it cannot be modified in-place, once stored in a certain memory location. You can retrieve any character from the string with the help of its index, but you cannot replace it with another character.

Example

In the following example, character Y is at index 7 in HELLO PYTHON. Try to replace Y with y and see what happens.

```
var="HELLO PYTHON"
var[7]="y"
print (var)
```

It will produce the following **output** –

```
Traceback (most recent call last):
  File "C:\Users\users\example.py", line 2, in <module>
    var[7]="y"
```

```
~~~~^
```

```
TypeError: 'str' object does not support item assignment
```

The `TypeError` is because the string is immutable.

Python String Slicing

Python defines ":" as string slicing operator. It returns a substring from the original string. Its general usage is as follows –

```
substr=var[x:y]
```

The ":" operator needs two integer operands (both of which may be omitted, as we shall see in subsequent examples). The first operand `x` is the index of the first character of the desired slice. The second operand `y` is the index of the character next to the last in the desired string. So `var(x:y]` separates characters from `x`th position to `(y-1)`th position from the original string.

Example

```
var="HELLO PYTHON"

print ("var:",var)
print ("var[3:8]:", var[3:8])
```

It will produce the following **output** –

```
var: HELLO PYTHON
var[3:8]: LO PY
```

Python String Slicing With Negative Indexing

Like positive indexes, negative indexes can also be used for slicing.

Example

The below example shows how to slice a string using negative indexes.

```

var="HELLO PYTHON"
print ("var:",var)
print ("var[3:8]:", var[3:8])
print ("var[-9:-4]:", var[-9:-4])

```

It will produce the following **output** –

```

var: HELLO PYTHON
var[3:8]: LO PY
var[-9:-4]: LO PY

```

Default Values of Indexes with String Slicing

Both the operands for Python's Slice operator are optional. The first operand defaults to zero, which means if we do not give the first operand, the slice starts of character at 0th index, i.e. the first character. It slices the leftmost substring up to "y-1" characters.

Example

In this example, we are performing slice operation using default values.

```

var="HELLO PYTHON"
print ("var:",var)
print ("var[0:5]:", var[0:5])
print ("var[:5]:", var[:5])

```

It will produce the following **output** –

```

var: HELLO PYTHON
var[0:5]: HELLO
var[:5]: HELLO

```

Example

Similarly, y operand is also optional. By default, it is "-1", which means the string will be sliced from the xth position up to the end of string.

```
var="HELLO PYTHON"
print ("var:",var)
print ("var[6:12]:", var[6:12])
print ("var[6:]:", var[6:])
```

It will produce the following output –

```
var: HELLO PYTHON
var[6:12]: PYTHON
var[6:]: PYTHON
```

Example

Naturally, if both the operands are not used, the slice will be equal to the original string. That's because "x" is 0, and "y" is the last index+1 (or -1) by default.

```
var="HELLO PYTHON"
print ("var:",var)
print ("var[0:12]:", var[0:12])
print ("var[:]:", var[:])
```

It will produce the following **output** –

```
var: HELLO PYTHON
var[0:12]: HELLO PYTHON
var[:]: HELLO PYTHON
```

Example

The left operand must be smaller than the operand on right, for getting a substring of the original string. Python doesn't raise any error, if the left operand is greater, but returns a null string.

```

var="HELLO PYTHON"
print ("var:",var)
print ("var[-1:7]:", var[-1:7])
print ("var[7:0]:", var[7:0])

```

It will produce the following **output** –

```

var: HELLO PYTHON
var[-1:7]:
var[7:0]:

```

Return Type of String Slicing

Slicing returns a new string. You can very well perform string operations like concatenation, or slicing on the sliced string.

Example

```

var="HELLO PYTHON"

print ("var:",var)
print ("var[:6][:2]:", var[:6][:2])

var1=var[:6]
print ("slice:", var1)
print ("var1[:2]:", var1[:2])

```

It will produce the following **output** –

```

var: HELLO PYTHON
var[:6][:2]: HE

```

```
slice: HELLO  
var1[:2]: HE
```

TOP TUTORIALS

Python Tutorial
Java Tutorial
C++ Tutorial
C Programming Tutorial
C# Tutorial
PHP Tutorial
R Tutorial
HTML Tutorial
CSS Tutorial
JavaScript Tutorial
SQL Tutorial

TRENDING TECHNOLOGIES

Cloud Computing Tutorial
Amazon Web Services Tutorial
Microsoft Azure Tutorial
Git Tutorial
Ethical Hacking Tutorial
Docker Tutorial
Kubernetes Tutorial
DSA Tutorial
Spring Boot Tutorial
SDLC Tutorial
Unix Tutorial

CERTIFICATIONS

Business Analytics Certification
Java & Spring Boot Advanced Certification
Data Science Advanced Certification

Cloud Computing And DevOps
Advanced Certification In Business Analytics
Artificial Intelligence And Machine Learning
DevOps Certification
Game Development Certification
Front-End Developer Certification
AWS Certification Training



Chapters ▾

Categories ≡

COMPILERS & EDITORS

Online Java Compiler
Online Python Compiler
Online Go Compiler
Online C Compiler
Online C++ Compiler
Online C# Compiler
Online PHP Compiler
Online MATLAB Compiler
Online Bash Terminal
Online SQL Compiler
Online Html Editor

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |
[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2026. All Rights Reserved.